

scripts/check-constitution.sh — User Guide

Last verified: 2026-08-25 (§6.N/O/P/Q propagation-target floor, from the §6.N.2 bluff hunt of 2026-08-23; §6.H tracker exemption, LVA-134) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate)

Overview

Stub doc generated from the script's in-source header comment. See scripts/check-constitution.sh for canonical behavior. This stub exists so the CM-SCRIPT-DOCS-SYNC pre-push gate (.github/hooks/pre-push Check 9) starts gating modifications to this script.

The script's own header documentation (verbatim):

```
scripts/check-constitution.sh – verify constitutional clauses present.
```

Per the SP-3a plan Task 5.19. Asserts that the three SP-3a clauses (6.D, 6.E, 6.F) are present in root CLAUDE.md and that the submodules/tracker_sdk/CLAUDE.md exists. Run from scripts/ci.sh in every mode.

Usage

See the script's in-source comment block (above) for canonical usage examples.

Clauses verified (current set, growing list)

The script asserts presence of the following constitutional clauses + supporting infrastructure:

- §6.D + §6.E + §6.F (root CLAUDE.md)
- submodules/tracker_sdk/CLAUDE.md exists
- core/CLAUDE.md references §6.E
- feature/CLAUDE.md references Challenge Test requirement

- §6.H credential pattern absence (no plaintext credentials in tracked files)
- §6.K Containers extension presence
- §6.X Container-Submodule Emulator Wiring inheritance + runtime checks (a) + (b)
- **§6.AD HelixConstitution Inheritance** — clause + constitution submodule + 54 per-scope inheritance pointer-blocks present
- **§6.W remote-host boundary** — only github + gitlab named remotes on parent + Lava-owned submodules
- **§11.4.6 no-guessing vocabulary** — forbidden words in tracked status/closure files unless prefixed by UNCONFIRMED:/UNKNOWN:/PENDING_FORENSICS:
- **§6.AE Comprehensive Challenge Coverage + Container/QEMU Matrix Mandate** (added 2026-05-15) — clause + scripts/check-challenge-coverage.sh + scripts/run-challenge-matrix.sh exist + executable

Maintenance

When this script is modified, update this document in the same commit (CM-SCRIPT-DOCS-SYNC requires it). Per §11.4.18, the documentation **MUST** stay in sync with the codebase — no doc may be out of sync with its script.

Cross-references

- scripts/check-constitution.sh — the script itself
- docs/helix-constitution-gates.md — gate inventory
- HelixConstitution Constitution.md §11.4.18 (the mandate)
- Lava CLAUDE.md §6.AD (HelixConstitution Inheritance)

2026-05-15 update — HelixQA waiver

Phase 4 of the constitution-compliance plan adopted submodules/helixqa (HelixDevelopment-owned QA orchestration framework) at upstream HEAD 403603db. HelixQA's CLAUDE.md / AGENTS.md / CONSTITUTION.md follow the canonical-root ## INHERITED FROM Helix Constitution pointer pattern (HelixDevelopment-authored) rather than Lava's heading-anchored §6.R / §6.S / §6.X / §6.AD pointer-block format. HelixQA also lacks helix-deps.yaml + install_upstreams.sh wrapper script.

Resolution: HELIX_DEV_OWNED=("HelixQA") waiver list + is_helix_dev_owned() helper skip HelixQA in every per-Submodule loop in this scanner. Waiver entries cite Phase 4-debt: PR to

HelixDevelopment/HelixQA upstream owed to add the missing files. Once upstream merges + Lava's pin advances to include them, HelixQA can be removed from the waiver list.

2026-05-16 update — HelixQA waiver RESOLVED

The Phase 4-debt PR to HelixDevelopment/HelixQA upstream landed as commit b13ba7c (feat(gov): add helix-deps.yaml + install_upstreams.sh wrapper), adding both missing files at HelixQA's repo root. Lava's pin advanced to b13ba7c in the same parent commit that removed HelixQA from the Lava-side waiver lists in scripts/check-helix-deps-manifest.sh + scripts/check-canonical-root-and-upstreams.sh. Both downstream scanners (CM-HELIX-DEPS-MANIFEST + CM-CANONICAL-ROOT-CLARITY + CM-INSTALL-UPSTREAMS-RAN gates) now treat HelixQA on equal terms with the other 16 owned submodules — 17/17 own-org submodules satisfy the constitutional surface; 0 waived. This scanner (scripts/check-constitution.sh) did not directly carry a HelixQA waiver itself — the waiver lived in the two downstream scanners it transitively invokes via scripts/verify-all-constitution-rules.sh. The waiver-state synchronization across the three docs is maintained per §11.4.18 (script-doc sync).

2026-07-02 update — §6.N/O/P/Q propagation-check fix (snake_case glob + pointer-block acceptance)

Regression found: the §6.N / §6.O / §6.P / §6.Q propagation blocks (9 / 9b / 9d / 9e) built their propagation_targets list from the **pre-migration CamelCase** submodule directory names (Auth, Cache, Tracker-SDK, ... — the literal submodules/<CamelCase>/CLAUDE.md path form). After the §11.4.29 snake_case migration renamed every submodule dir to lowercase (submodules/auth/, submodules/tracker_sdk/, ...), those old paths stopped resolving, so the per-target [[! -f "\$f"]] && continue guard **silently skipped every submodule** — turning all four propagation gates into no-ops for submodules. The drift went unnoticed because the parallel §6.AD pointer-block mechanism was doing the actual inheritance. Surfaced by the hermetic tests/check-constitution/check_constitution_test.sh::test_missing_6n_from_submodule_fails, which was failing (the checker passed a fixture with §6.N stripped from submodules/auth/CLAUDE.md).

Fix (two parts, no submodule backfill):

1. `propagation_targets` now enumerates submodule docs via a migration-proof `submodules/*/CLAUDE.md` glob instead of the CamelCase literal list.
2. Blocks 9 / 9b / 9d / 9e now use the existing `doc_inherits_clause "$f" "6.N" helper` (literal clause **OR** the `## INHERITED FROM constitution/ §6.AD pointer-block`) — the *same* mechanism the §6.R / §6.S / §6.X propagation gates already accept. This is provably **not** a weakening: block 6.AD(4) independently HARD-enforces the pointer-block's presence in every submodule doc, so accepting it here adds no pass-path that isn't already guaranteed and gated.

Why not backfill legacy per-clause sections into 16

submodules: every submodule already carries the §6.AD pointer-block (the canonical post-2026-05-14 inheritance mechanism), which transitively inherits §6.N/O/P/Q. The explicit `## Clause 6.N` sections that only `http3/mdns/tracker_sdk` carry are pre-§6.AD "Group A" legacy; duplicating them into every submodule would be redundant churn, not real coverage.

The companion test was updated to strip BOTH the literal §6.N and the pointer-block from `submodules/auth/CLAUDE.md`, so it faithfully exercises the block-9 rejection path (a submodule with NO §6.N inheritance mechanism).

2026-08-23 update — §6.N/O/P/Q propagation-target FLOOR (finding F3)

Added by the §6.N.2 gate-shaping bluff hunt. The change is small — one counter, one `if`, and the loop body rewritten from an `&& list` to `|| continue` — and it closes the second of two routes into the same no-op.

The defect. Block 9 seeds `propagation_targets` with 5 root docs (`CLAUDE.md`, `AGENTS.md`, `lava-api-go/CLAUDE.md`, `lava-api-go/AGENTS.md`, `lava-api-go/CONSTITUTION.md`) and then extends it from a glob over `submodules/*/CLAUDE.md`. `nullglob` is not set in this script, so when that glob matches nothing bash leaves the **literal string** `submodules/*/CLAUDE.md` as the loop's single value. `[[ -f ... ]]` is false for it, and nothing is appended.

The old body was `[[ -f "$sub" ]] && propagation_targets+=("$sub")`. A failing `[[ ... ]]` there is the *non-final* command of an `&& list`, a position set `-e` explicitly exempts — so the script did not abort. It continued, blocks 9 / 9b / 9d / 9e examined only the 5 root docs, and the gate reported **PASS**.

The consequence is that one repository state produced two opposite verdicts, decided by something unrelated to the property under test:

Checkout	What block 9 examined	Verdict on identical drift
Submodules initialised	5 root docs + 17+ submodule docs	drift caught, exit 1
Submodules uninitialised	5 root docs only	PASS , exit 0

A gate reporting "*nothing failed*" when it means "*nothing was learned*" is the shape §6.J forbids. This same file already guards against exactly that shape one block earlier: the §6.H credential scan refuses to pass on an empty corpus ("clause 6.H credential scan examined ZERO tracked files." ... "a PASS here would assert nothing.", lines 150–151). The propagation gate simply had no equivalent.

Not hypothetical. The block's own comment records the identical no-op having already happened once: the §11.4.29 `snake_case` migration renamed every submodule directory to lowercase, the hard-coded CamelCase paths stopped resolving, and the guard "*silently skipped EVERY submodule ... hiding real propagation drift*" until 2026-07-02. That fix replaced the CamelCase literals with the glob — it closed the rename route and left the uninitialised-checkout route into the same no-op open. A fresh git clone without `--recursive` is the ordinary way to arrive there.

The fix. The submodule contribution is counted, and a count of zero is a hard failure with an actionable remedy rather than a pass:

```
propagation gate examined ZERO submodule CLAUDE.md files.  
→ submodules/*/CLAUDE.md matched nothing, so the §6.N/§6.O/§6.P/  
§6.Q  
    propagation blocks would check only the 5 root docs.  
→ A PASS here would assert nothing about submodule propagation.  
→ Initialise the submodules and re-run:  
    git submodule update --init --recursive  
    (scripts/setup-clone.sh reports the same remedy at its step  
4.)
```

Exit status is **1**. The floor can only fire when the gate was about to make an unbacked claim: a real checkout carries 17+ own-org submodules, so a correctly initialised tree never reaches it. Operators who hit it have an uninitialised clone, and the message names the one command that fixes it.

Regression coverage. tests/check-constitution/test_propagation_target_floor.sh. Like the other gate-shaping tests from this hunt it **extracts the block under test verbatim** from the live script (two marker-anchored ranges: the doc_inherits_clause() helper, and declare -a propagation_targets= through the second done), so it tracks the shipped code instead of a copy that could keep passing after the real one regressed. A verify_extraction guard fails loudly if those anchors move, because assertions against an empty harness would be vacuous — a bluff by construction.

Four cases, deliberately paired so that a blanket fail-everything change cannot satisfy the suite:

#	Kind	Fixture	Asserted
1	regression	5 compliant root docs, submodules/empty	non-zero exit — the gate must not pass having examined zero submodules
2	regression	same	the failure names git submodule update --init as the remedy (a fresh clone is how operators arrive here; an opaque failure would not be actionable)
3	control	3 compliant submodules present	exit 0 — the floor does not break the legitimate initialised workflow
4	control	3 submodules, submodules/cache/CLAUDE.md drifted	non-zero exit naming propagation REGRESSED: submodules/cache/CLAUDE.md — the floor did not replace or mask the real propagation check

Cases 1 and 2 fail against the pre-fix code and pass against the fix; cases 3 and 4 pass against both, which is what makes them controls. The file is picked up by scripts/verify-all-constitution-rules.sh's tests/check-constitution/test_*.sh discovery loop, so it is a registered gate of the sweep.

2026-08-25 update — §6.H credential-scan exemption for the generated trackers (LVA-134)

Landed alongside the propagation floor above, from a separate work stream. The change is one alternation added to the corpus-exclusion regex plus the rationale recorded in the source:

...|^((docs/)?(Issues|Fixed)(_Summary)?\.(md|html|pdf|docx))\$

Why. Every exemption already in that list is the same category — a surface whose purpose is to **describe** the forbidden pattern rather than to use a credential: `.env.example` (a template), `CHANGELOG.md` (historical incidents), `.lava-ci-evidence/` (forensic leak records), `docs/INCIDENT_*`, the governance docs, and this scanner's own source. The generated issue tracker was the one such prose surface missing from the list, and it is the surface *most* likely to describe a credential-pattern defect. LVA-134 — the ticket filed about this very false positive — quotes the pattern verbatim in its body, so regenerating the trackers from `docs/workable_items.db` reproduced the violation in four more tracked files. Filing the bug re-created the bug.

Scope corrected 2026-08-26. The anchor named `docs/` only, but this repo tracks the SAME generated renderings at the repository **root** as well — 16 paths in each location, and the two copies are byte-identical (verified with `cmp` on all four `.md` and all four `.html` members). The root copies were therefore still scanned, and the next tracker regeneration reproduced the LVA-134 body's verbatim pattern in `Fixed.md`, `Fixed.html`, `Fixed_Summary.md` and `Fixed_Summary.html` at the root. Measured:
`git show HEAD:Fixed.md | grep -c = 0`, working tree = 1 — the hit was NEW generator output, not inherited content. The `(docs/)?` is not a loosening: it finishes covering the surface the exemption was written for. Exempt total goes from 16 files to 32 (0.58 % of 5510 tracked), and the root half is the same bytes as the already-exempt `docs/` half, so no additional information is withheld from the scan.

Why all four names, not just the two that were red. The trackers are generated: an OPEN item renders into `Issues*`, and closing it MOVES the same body into `Fixed*`. Exempting only `Issues*` would re-break this gate at the moment LVA-134 is marked closed. The scope follows from that mechanic, not from which files happened to be failing on the day.

Coverage cost, stated plainly (\$6.J — an exemption is surface no longer scanned). Independently measured against this checkout:

Quantity	Measured
Tracked files newly exempted	16
Tracked files in the repository	5510
Share of corpus removed	0.29%

All 16 are generated from `docs/workable_items.db`; none is handwritten source. The `.pdf / .docx` members were already effectively unscanned because `grep -I` skips binaries, so the **real** new loss is 8 files: `{Issues,Fixed}{,_Summary}.{md,html}`. Residual

risk: a real credential pasted into a ticket body would no longer be caught in its markdown or HTML rendering. That is accepted because the rendering is a copy — the authoritative store, docs/workable_items.db, is binary and was never scanned by this block.

What was deliberately NOT exempted: tests/.** The hermetic fixture in tests/check-constitution/test_credential_scan_corpus.sh that used to trip this scan now **assembles its leak string at runtime** (printf '%s object CredsBridge {\n}\n' 'private', line 117) — the same self-safe idiom scripts/scan-no-hardcoded-ipv4.sh uses on itself. All 102 tracked files under tests/ therefore remain fully inside the scan corpus.

That choice is the load-bearing one. The two sibling \$6.R scanners (scan-no-hardcoded-ipv4.sh, scan-no-hardcoded-hostport.sh) blanket-exempt ^tests/; copying that here would have cost **102 files** of coverage to fix **one** file's false positive. Runtime assembly costs nothing and is self-verifying: if the assembly ever drifts so the written file no longer carries the pattern, the fixture stops reproducing the leak and the test fails rather than passing vacuously.

Anti-bluff verification of the exemption (executed, not asserted)

An exemption is coverage removed, so the exemption itself is gated by a new fourth case in tests/check-constitution/test_credential_scan_corpus.sh:

- **test_exemption_does_not_overreach** plants a leak in docs/Issues_Notes_*.md — a path that *begins* with docs/Issues and *ends* in .md, so an unanchored or merely sloppier exemption regex would swallow it — and asserts the scanner still reports a clause-6.H violation. The anchoring of ^(docs/)?(Issues|Fixed)(_Summary)?\.(md|html|pdf|docx)\$ is what makes that file scanned rather than exempt.
- **test_root_exemption_does_not_overreach** (2026-08-26) is the same probe at the repository ROOT, and **test_root_tracker_rendering_is_exempt** proves the root renderings really are covered — so the newly-optional docs/ component cannot silently become a prefix sweep in either direction.

Three mutations were rehearsed against the landed change. Each produced a failure, and each was reverted with the file restored byte-identically (sha256-verified):

Mutation	Effect on the system	Observed
A — assembled keyword private → public at both call sites	the fixture writes a file that no longer carries the pattern	FAIL test_real_violation_still_caught
B — exemption widened from the anchored form to a blanket ^docs/	test 4's fixture is swallowed, emptying the corpus	FAIL test_exemption_does_not_overreach (the empty-corpus guard fired)
C — the Bridge pattern deleted from forbidden_credential_patterns	the scanner is blinded to the C2 shape entirely	FAIL test_real_violation_still_caught

Mutation C is the load-bearing one: it confirms the fix did not blind the scanner, because removing real detection still fails the suite.

Independently of the hermetic suite, genuine credential-shaped literals were planted in two clean tracked files that are **not** exempt — README.md (the private object ...Bridge { shape) and scripts/ci.sh (a literal RUTRACKER_PASSWORD assignment to a quoted string constant) — and the scanner caught **both** after the fix, reporting 2 clause-6.H violations. Both files were then restored and verified byte-identical to their pre-plant sha256, leaving no working-tree trace.

Ordering consequence worth keeping in mind

Block 6 sits ahead of the §6.N/§6.O/§6.P/§6.Q propagation blocks and of every gate after them. Any exit 1 there is not a local failure — it truncates the run. Measured on this checkout: the failing run emitted **0** ✓ markers; the fixed run reaches ✓ clause 6.K and continues. When adding future patterns to this block, weigh that a false positive here silently costs the rest of the gate, not just its own check.

§6.H credential-scan corpus floor (added 2026-08-22)

The clause-6.H credential scan builds `tracked_files` from `git ls-files` and then greps each entry for credential patterns. Before this floor existed, a `git ls-files` that yielded **nothing** — a broken index, a non-repo working directory, a stubbed or failed git — made the scan loop iterate zero times, report `credential_violations=0`, and print "no clause-6.H credential patterns in tracked files".

That is "*nothing was learned*" reported as "*nothing failed*": the exact shape §6.J forbids. The `|| true` on the collecting pipeline — needed because `grep -v` exits 1 on an all-filtered corpus — is what made the failure silent rather than loud.

The gate now refuses outright when the corpus is empty:

```
clause 6.H credential scan examined ZERO tracked files.  
  → The scan corpus is empty, so a PASS here would assert nothing.  
  → Check that 'git ls-files' works in <cwd> (repo present, index  
     readable,  
     git on PATH) and re-run.
```

A healthy checkout has thousands of tracked files, so this guard **cannot fire on a real run** — it fires only when the gate is about to make an unbacked claim. That property is what makes it safe to add to a gate that runs on every push.

Related: the same empty-corpus shape was found at 20 further sites during the 2026-08-26 §6.N.2 sweep and closed with this idiom. See also the propagation-target floor below, which additionally distinguishes an *uninitialised* submodule corpus from genuine propagation drift, because the two call for opposite remedies.